

Lab 5 - Reinforcement Learning

Group 3 - Subgroup 13

November 12, 2024

Part II: Q-Learning Angle Controller

Question 1 (theory):

- a)
 - For the state, we discretize the angle into 20 pieces between all possible values that the angle can have $[-\pi, \pi]$. This was done to make sure we cover information about the whole environment while exploring. We tried different "number of values" for the discretization, and chose 20 because it worked well.
 - For the reward, since the goal is to keep the rocket straight with the an angle as close to zero as possible, any angle away from zero is bad. We make the penalty stronger the further away from zero the angle is by squaring the angle value. This means the rocket really tries to avoid being tilted too much.
 - For the actions, we used all possible combinations of actions (8 in total).
- b)
 - **Q-values:** Estimates the usefulness of taking a certain action from a specific state, guiding the system to make decisions that maximize future rewards.
 - **Learning Rate** (α): Determines how new information affects accumulated knowledge. A higher learning rate means the algorithm quickly adopts the new data, while a lower rate means it relies more heavily on old data.
 - **Discount Factor** (γ): Reflects the importance of future rewards. A higher discount factor makes the agent prioritize long-term gains more strongly.

Question 2: If exploration is disabled too early, the agent will not have the opportunity to explore different actions and will tend to repeat

the same initial random actions. Before any updates, all action values (Q-values) in the table will be set to zero. As the agent starts interacting with the environment, it will update these values based on the rewards it receives. But if the agent doesn't explore different states, it might mistakenly conclude that the current table includes the best actions. This is because without trying out other options, the agent has no way to learn if other actions might lead to better rewards. So it might end up with a very limited and possibly incorrect understanding of what actions are truly optimal in various situations.

Part III: The Full Q-Learning Hover Controller

- For the state in the full hover controller:
 - We discretize the angle into 20 bins across the range $[-\pi, \pi]$ to cover information about the whole environment when it comes to the angle as done in the angle controller part.
 - The vertical velocity into 8 bins from $[-4, 4]$.
 - The horizontal velocity into 5 bins from $[-1.5, 1.5]$.

The discretization of vertical velocity used more bins compared to horizontal velocity due to the greater importance of controlling vertical movement for hovering. Horizontal drift was not as significant during the angle controller part of the lab, so fewer bins for horizontal velocity were sufficient

- For the reward, we use a combination of penalties:
 - **Angle:** Cubic penalty to heavily penalize deviations from the center.
 - **Vertical Velocity (vy):** Quadratic penalty multiplied with a constant (1000) to ensure the rocket doesn't drift too far vertically.
 - **Horizontal Velocity (vx):** Also a quadratic penalty but less severe than vy, multiplied with a smaller constant (700). This was done because there weren't much horizontal drift noticed in the angle controller part.

The parameter values for states and values presented above were mostly determined through experimenting and testing different values. Our goal was to maintain simplicity with a manageable number of states while penalizing undesirable outcomes that deviate a lot. For a perfect hover, we need to minimize larger deviations in velocities to avoid unwanted acceleration.

- In the last part, we also experimented with different numbers of actions. In the angle controller part, we used all possible combinations of actions (8 in total). We noticed that using all these actions, along with all the states for the hovering part, caused the agent to take a long time to learn to do a good hover (about 15 minutes). When we reduced the number of actions to 5, the time required for achieving good hover decreased to about 5 minutes. With fewer actions, the Q-learning algorithm had to manage a smaller set of possibilities. As a result, training time decreased, and the rocket achieved stable hover more quickly compared to the setup with more actions. However, the setup with all 8 actions which took longer to train, resulted in a more precise and perfect hover compared to the faster learning setup with fewer actions.